Coursework 1 (Edinburgh CW1 15)

F29AI - Artificial Intelligence and Intelligent Agents

Authors :

- Léandre Ramos - **H00550789** - lgr4000@hw.ac.uk
- Lohith Viswa Pentakota - **H00521065** - lp4015@hw.ac.uk

Video : Demo-link

Use of Generative AI. *We used Claude (Anthropic) to help us understand concepts in English (not our first language), and to find Sudoku test puzzles from public sources.*
Also we use grammarly to fix grammar. The core logic, code, the solver, the pddl files, all the design decisions are our own.

Part 1A : Sudoku as a CSP

A Sudoku can be set up as a constraint satisfaction problem (CSP) with the help of the following formulation [1, Sec. 6.2.6].

Notation. The grid has 9 rows, named A to I, and 9 columns, numbered 1 to 9. Each cell is named by its row and its column, from A1 to I9.

Variables. There is one variable per cell, so 81 variables, from A1 to I9. Empty cells and prefilled cells are both variables: the only difference between them is their domain.

Domains. An empty cell can take any value in {1, 2, 3, 4, 5, 6, 7, 8, 9}. A prefilled cell has a domain with only one value, the given digit, which cannot change during the solving.

Constraints. Each cell belongs to one row, one column and one 3x3 square. A square is made of three consecutive rows (A-C, D-F or G-I) and three consecutive columns (1-3, 4-6 or 7-9), which gives 9 squares. A value can only be placed in a cell if it is not already used in the same row, column or square. With 9 rows, 9 columns and 9 squares, this gives a total of 27 Alldiff constraints [1, Sec. 6.2.6].

A **solution** is a complete assignment of the 81 variables that satisfies every constraint.

Let m be the number of empty cells.

Brute force tries every possible way to fill the empty cells, then checks each complete grid. With 9 possible digits per cell, there are 9^m grids, so the worst case is O(9^m). A Sudoku with a unique solution has at least 17 givens [2], so m can be up to 64. This gives 9^64, about 10^61 grids, which is impossible to test in practice.

Backtracking is a depth-first search: it fills one cell at a time and goes back as soon as a constraint is broken [1, Sec. 6.3]. In the worst case, it can still explore almost the whole search tree, which has 9^m leaves [1, Sec. 6.3], so it is also O(9^m). In practice, it is much faster thanks to pruning: if a digit breaks a constraint at depth i, the whole branch below is cut, and the 9^(m-i) grids it contains are never generated. Moreover, each cell has 20 neighbours (8 in its row, 8 in its column and 4 more in its square), so most cells have far fewer than 9 possible digits. Backtracking therefore only explores a very small part of the 9^m grids.

Part 1B : Implementation of Sudoku

1. Our approach

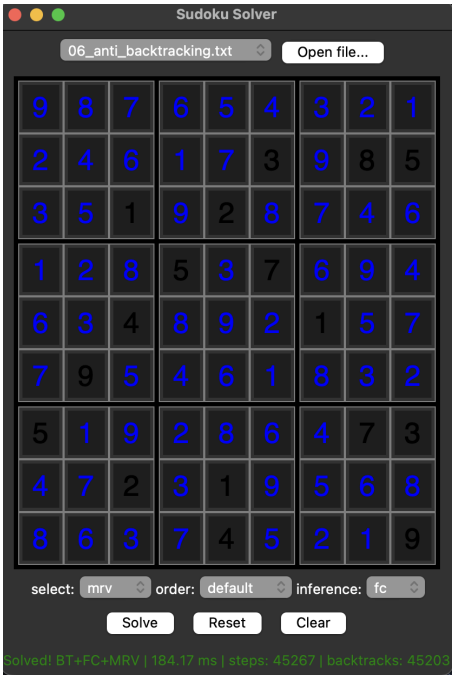
To solve a Sudoku, we chose a simple CSP approach: backtracking search, improved with two techniques, Minimum Remaining Values (MRV) and Forward Checking (FC). Each one answers a different question. MRV answers "which variable should be assigned next?": it chooses, among all the empty cells, the one with the fewest possible values. FC answers "can we detect inevitable failure early?": after each assignment, it removes the value from the domains of the empty neighbours, and goes back as soon as one of them has no value

left. The two techniques work well together, because FC keeps the domains up to date and MRV uses these updated domains to choose the most constrained variable.

2. Implementation

The solver is written in Python with no external library. It reads a `.txt` or `.csv` file, prints the solved grid, or prints "No solution". We also developed a GUI to test the different approaches easily (Figure 1): three menus choose which variable to fill (first empty cell, MRV or MRV advanced), which value to try first (1 to 9, or LCV) and the inference (none or FC).

[Figure 1: The GUI after solving puzzle 06 with BT+FC+MRV.]



3. Experimental setup

We tested our approach on 8 puzzles with different particularities (Table 1), from easy puzzles to a puzzle built against backtracking, and two unsolvable puzzles. For each run, we measured three values: the time in ms, the number of steps (digits tried, i.e. nodes of the search tree) and the number of backtracks (digits undone).

To evaluate the quality of our approach, we compared it with other versions: plain backtracking (BT), backtracking with forward checking only (BT+FC), our approach (BT+FC+MRV), and two extensions. The first one, MRV adv, breaks the MRV ties with the degree heuristic (BT+FC+MRV adv). The second one adds Least Constraining Value (BT+FC+MRV+LCV), which answers a third question: in what order should the values be tried? The steps and backtracks are deterministic, so they are the same at every run. Only the time changes a little, so we ran the benchmark three times and kept the average time. All runs were made on a MacBook Pro M4 Pro, so the times can be different on another machine.

Table 1: Test set and results of our solver (BT+FC+MRV).

Puzzle	Givens	Type	Source	Result	Time (ms)	Steps	Backtracks
01	30	easy	[4]	solved	0.22	51	0
02	32	easy	[5]	solved	0.21	49	0
03	17	hard for computers	[6]	solved	1.85	481	417
04	17	hard for computers	[6]	solved	6.86	1,791	1,727
05	21	hard for humans	[7]	solved	36.98	10,101	10,041
06	17	against backtracking	[3]	solved	160.09	45,267	45,203
07	9	no solution	ours	no solution	0.07	0	0
08	30	invalid givens	ours	no solution	0.00	0	0

Table 2: Steps for each method (best in bold).

Puzzle	BT	BT+FC	BT+FC+MRV (ours)	MRV adv	LCV
01	13,085	516	**51**	**51**	**51**
02	552	95	**49**	**49**	**49**
03	45,232,227	1,715,483	481	2,090	**111**
04	16,257,319	1,194,672	1,791	2,551	**1,599**
05	191,505	22,067	10,101	**9,171**	9,264
06	310,190,708	20,872,634	45,267	**13,971**	65,081

Table 3: Time in ms on the hard puzzles, average of 3 runs (best in bold).

Puzzle	BT	BT+FC	BT+FC+MRV (ours)	MRV adv	LCV
03	26,614	2,777	1.85	60.83	**0.61**
04	9,552	2,012	**6.86**	78.32	8.18
05	124.57	**35.01**	36.98	287.84	44.73
06	184,118	35,440	**160.09**	424.63	319.47

4. Analysis
As we expected, each technique removes a large part of the search. FC alone divides the number of steps by 6 to 26 (Table 2), and MRV makes the biggest difference on hard puzzles: on puzzle 03, the steps go from 45 million with plain BT to 481 with our approach, and the time from 26.6 s to 1.85 ms (Table 3). This supports the argument of Part 1A: pruning is what makes backtracking efficient in practice, and FC and MRV make it happen earlier in the tree. On easy puzzles, our approach never makes a wrong choice: puzzle 01 needs exactly 51 steps for its 51 empty cells, with 0 backtracks, because after each forward check MRV always finds a cell with only one possible value. The two unsolvable puzzles are detected before any step: puzzle 07 has a cell with an empty domain from the start, and puzzle 08 already breaks a constraint.

A less expected result is that a puzzle that is difficult for humans is not always difficult for a CSP approach. Puzzle 05 was presented as the hardest Sudoku in the world for humans [7], but plain BT solves it in 191,505 steps, far fewer than the 45 million steps of puzzle 03. Puzzle 06 is the hardest for every method, because it was built for this: the first row of its solution is 9 8 7 6 5 4 3 2 1, while backtracking tries the values from 1 to 9 [3].

What surprised us most is that exploring fewer nodes does not always mean solving faster. MRV adv makes 3 times fewer steps than our approach on puzzle 06, but it is 2.6 times slower, because counting the empty neighbours of every candidate costs about 30 µs per step, against 3.5 µs for MRV. In the same way, LCV is the best on puzzle 03, but the worst extension on puzzle 06. No technique is the best on every puzzle.

5. Strengths and limitations
Our approach seems to be the most convenient in terms of search and time: it solves every puzzle in less than 0.2 s and detects the unsolvable ones. Even if it does not always explore the fewest nodes, it is the fastest on 4 of the 6 solvable puzzles and has the lowest total time (206 ms, against 374 ms for LCV and 854 ms for MRV adv), because each step needs fewer computations.

However, the worst case is still O(9^m), and a puzzle built against backtracking still needs 45,267 steps. FC only looks at the direct neighbours of the assigned cell. Arc consistency (AC-3) would propagate the constraints further: on its own, it only solves the easiest Sudokus [1, Sec. 6.2.6], but the inference step of backtracking can also apply it instead of FC [1, Sec. 6.3]. We would expect it to explore fewer nodes, but with a higher cost per node, as we saw with MRV adv. Human solvers also use more complex inference rules, such as naked triples [1, Sec. 6.2.6]. Finally, our test set is small and our times depend on the machine.

6. A* analysis
Like any CSP, Sudoku can be formulated as a standard search problem [1, Sec. 6.3], and solved with A*:

- **State:** a partially filled grid that respects the constraints. The initial state is the puzzle.
- **Actions:** write a consistent digit in one empty cell. The cell is always chosen with the same rule (for example MRV), otherwise the same state can be reached in many different orders.
- **Goal test:** the grid has no empty cell.
- **Path cost:** 1 for each filled cell, so every solution costs m.
- **Heuristic:** h(n) = number of empty cells. It is admissible, because it is exactly the remaining cost when n can lead to a solution. We can also set h(n) = infinity when a cell has no possible value, which removes dead ends.

However, A* is not well suited to Sudoku. With this formulation, f(n) = g(n) + h(n) = (filled cells) + (empty cells) = m for every state, so A* cannot rank the states. A* is designed to find the cheapest path, but in Sudoku every solution has the same cost: we only need one valid grid, so it is a satisfaction problem, not an optimisation problem. A* also stores the whole frontier in memory, which grows exponentially, while backtracking keeps a single grid and modifies it. To be efficient, A* would need constraint propagation and MRV, and it would then become our approach with an extra priority queue. For these reasons, backtracking with FC and MRV is a better choice for Sudoku than A*.

Part 2: Automated Planning with PDDL

1. Overview and planners

For Part 2 we wrote two domains and three problem files. The first domain, lunar, is used for Missions 1 and 2, and the second one, lunar-extended, adds the astronauts for Mission 3. We only needed :strips and :typing, which all the planners of the brief support [8].

Planners used. We ran each mission with BFWS (FF-parser version) and LAMA-first, the two planners recommended in the brief [9], [10]. LAMA-first is in fact a configuration of Fast Downward [11]. We also tried OPTIC [12], mainly to get a third opinion on our plans.

Landing choices. The brief lets us choose some landing sites, so we fixed each lander's waypoint in the initial state instead of writing a landing action. In Mission 1, Lander 1 is at WP1: the sample is already there, so the rover can collect it and store it without moving. In Missions 2 and 3, Lander 2 is at WP2, next to Lander 1. WP2 is connected to most of the map, so it is a good starting point for both rovers.

2. Domain design (Part 2A)

Our domain has seven object types, fifteen predicates and seven actions (Tables 4 to 6). All the actions use typed parameters and never name a specific waypoint, rover or lander, so the same file works for Mission 1, Mission 2 and any other mission that follows the specification.

Table 4: Object types

Type	Parent	Role
lander	none	Stationary base. Each rover belongs to exactly one lander.
rover	none	Moves, holds one item of data and carries one sample.
waypoint	none	A location on the map.
data	none	Anything a rover can transmit.
image, scan	data	Separate subtypes, so capture_image only accepts images and perform_scan only accepts scans.
sample	none	A physical sample. It is carried and stored, never transmitted.

Table 5: Predicates

Purpose	Predicates	What they represent
Position and map	rover_at, lander_at, connected	Where rovers and landers are. Each connected w1 w2 fact is one directed edge.
Deployment and ownership	deployed, onboard, owned_by	A rover is on the surface or still aboard its lander, and which lander it belongs to.
Data and memory	data_at, memory_free, carrying_data, saved_data	Where data can be captured, whether the rover's data slot is empty, what it holds, and what has been transmitted to a lander.
Samples	sample_at, sample_capacity_free, carrying_sample, lander_sample_capacity_free, stored_sample	Where samples are, the rover's and the lander's one-sample slots, what the rover carries and what is stored.

Table 6: Action schemas

Action	Key preconditions	Effects
move r from to	deployed, rover_at from, connected from to	Rover leaves from and is at to.
deploy r l w	owned_by r l, onboard r l, lander_at l w	No longer onboard. Now deployed and rover_at w.
capture_image, perform_scan	deployed, rover_at w, data_at d w, memory_free	Memory no longer free. Rover is carrying_data.
transmit_data r l d	deployed, owned_by r l, carrying_data r d	Stops carrying d. saved_data d l holds. Memory free again.
collect_sample r s w	deployed, rover_at w, sample_at s w, sample_capacity_free	Rover slot used. Sample leaves w. Rover is carrying_sample.
store_sample r s l w	deployed, owned_by r l, rover_at w, lander_at l w, carrying_sample, lander_sample_capacity_free	Rover slot free. Lander slot used. stored_sample s l holds.

2.1 Reasoning behind the less obvious choices

- **Deployment.** Before deployment, a rover is onboard its lander and has no rover_at fact, so it is not on the map. deploy is the only action that gives it a waypoint, the one where its own lander is, and every other rover action needs deployed.
- **Wireless transmit.** transmit_data has no waypoint in its preconditions: the rover only needs to be deployed, to hold the data and to own the lander. It can therefore transmit from anywhere on the map.
- **Rover and lander association.** owned_by appears in deploy, transmit_data and store_sample, so a rover can only work with its own lander. As each goal names a lander, in Mission 2 it is the goal that decides which rover can achieve it.
- **Lander mobility.** Landers have no action. lander_at is set once in the initial state and no effect removes it, so a lander never moves.
- **Directed connectivity.** A connected fact only allows travel in one direction. We added the reverse fact only where the brief's figures show a double arrow, so no edge is bidirectional by default.

2.2 Memory, sample capacity and storage

- **Rover memory.** memory_free is a single flag per rover. Capturing an image or doing a scan needs it and removes it, and only transmit_data gives it back, so a rover never holds more than one unsent item. Capturing only adds carrying_data, while the goal fact saved_data only comes from transmit_data: capturing alone is not enough to reach a goal. As transmit needs carrying_data and removes it, a rover cannot send data it never acquired, or send the same item twice.
- **Rover sample capacity.** sample_capacity_free plays the same role for samples. collect_sample needs it and removes it, and only store_sample gives it back, so the rover has to store its sample before collecting a new one. Collecting also removes sample_at, so the same sample cannot be picked up twice, and only stored_sample counts for the goal.

- **Lander storage.** lander_sample_capacity_free is the single slot of a lander. store_sample needs it and removes it, and no action gives it back, so a lander stores at most one sample. store_sample also uses the same w for rover_at and lander_at, which forces the rover to go to its own lander's waypoint.

3. Problem modelling (Part 2B)

Each problem file declares its objects, its initial state and its goals. The map, the landing sites and the status of the rovers are all in the initial state, so the domain never changes between missions. We copied the connections exactly as they are drawn in Figures 2 and 3 of the brief.

Table 7: Mission 1 and Mission 2 problem files

Item	Mission 1	Mission 2
Map	5 waypoints, 6 one-way edges (Figure 2)	6 waypoints, 9 directed edges: three two-way pairs and three one-way edges (Figure 3)
Landers and rovers	Lander 1 at WP1. Rover 1 aboard Lander 1.	Both landers at WP2. Rover 1 deployed at WP2. Rover 2 aboard Lander 2.
Data and samples	image1 at WP5, scan1 at WP3, sample1 at WP1	image1 at WP3, scan1 at WP6, image2 at WP2, scan2 at WP4, sample1 at WP5, sample2 at WP1
Other start facts	Memory and sample slots free for the rover and lander	Memory and sample slots free for both rovers and both landers
Goals	image1, scan1 saved to Lander 1. sample1 stored in Lander 1.	Lander 1: image1, scan1 saved, sample1 stored. Lander 2: image2, scan2 saved, sample2 stored.

Mission 2 is where the directed map really matters. WP3 only leads to WP5, so Rover 1 cannot come back the way it came: from WP5, its only way back to WP2 is through WP6 and WP4. The goals also split the work, as only Rover 2 can reach the goals of Lander 2. Mission 3 reuses this problem and adds the astronauts (Section 5).

4. Testing

All nine plans (three missions, three planners) were valid and reached every goal. We replayed each plan action by action against the domain, to check that every precondition was true before each action and that the final state contained all the goal facts. We also looked for:

- data transmitted before it was captured or scanned, or a second capture before a transmit;
- a sample not stored at its own lander's waypoint, or two samples carried at once;
- a move along an edge the map does not allow;
- Rover 2 acting before it was deployed (Missions 2 and 3), or an astronaut-gated action with no astronaut in the right room (Mission 3).

Table 8: Plan length in actions

Mission	BFWS	LAMA-first	OPTIC
1	10	10	10
2	27	21	22
3	32	25	27

- In the LAMA-first plan for Mission 1, the rover scans at WP3 but cannot take the image at WP5 straight away: its memory is full, so it first has to transmit the scan.
- In the LAMA-first plan for Mission 2, Rover 2 sends scan2 while it is still at WP4, far from its lander at WP2, which is exactly what wireless transmission should allow.
- The plans only use the edges of the map. In Mission 1, LAMA-first goes through WP1, WP4, WP3 and BFWS through WP1, WP2, WP3: both routes are legal.
- The plans do not all have the same length (Table 8). For Mission 1, the three planners agree on 10 actions, and we cannot do better: 7 actions are unavoidable and the rover

needs at least 3 moves. For Mission 2, the 21 actions of LAMA-first are also the minimum (13 unavoidable actions and 8 moves). BFWS needs 27, because it sends Rover 1 around the WP5 loop twice. Its plan is still valid, just not the shortest, since these planners stop at the first plan they find.

OPTIC is a temporal planner and our domain has no durations, so its timestamps do not mean anything here and we only compare the number of actions. As OPTIC is not one of the planners of the brief, we only used it as a cross-check, and our main results come from BFWS and LAMA-first.

5. Astronaut extension (Part 2C)

In lunar-extended, the position of the astronauts decides whether deploy, store_sample and transmit_data can be applied at all. These are real preconditions, so the planner has to move the astronauts before these actions.

What we added. Two types, astronaut and area, with control_room and docking_bay as subtypes of area, and three predicates: astronaut_at ?a ?area, astronaut_belongs ?a ?l and area_in ?area ?l. We also added one action, move_astronaut ?a ?from ?to ?l. It needs the astronaut to belong to l and both areas to be in l, so an astronaut can only move inside their own lander.

Table 9: Actions changed by the extension

Action	Extra parameters	Extra preconditions
deploy	?a - astronaut, ?db - docking_bay	astronaut_belongs a l, astronaut_at a db, area_in db l
store_sample	?a - astronaut, ?db - docking_bay	astronaut_belongs a l, astronaut_at a db, area_in db l
transmit_data	?a - astronaut, ?cr - control_room	astronaut_belongs a l, astronaut_at a cr, area_in cr l

The lander l is the same in owned_by, astronaut_belongs and area_in, so the astronaut and the room always belong to the rover's lander. Because the room parameter is typed, the planner cannot use the wrong kind of room either. The other actions did not change, so the extension fits directly on top of the Part 2A domain.

Mission 3 setup. The map, goals and landing sites are the same as in Mission 2. Alice belongs to Lander 1 and starts in docking_bay1, and Bob belongs to Lander 2 and starts in control_room2. Rover 1 is already deployed, so it never needs an astronaut for that.

How the constraints shape the plan. Compared to Mission 2, every planner needs 4 or 5 more actions (Table 8: 21 to 25, 27 to 32 and 22 to 27), and all of them are move_astronaut: the rover moves do not change. The LAMA-first plan shows why:

- Bob starts in the control room, so Rover 2 cannot be deployed yet. The plan starts with move_astronaut bob control_room2 docking_bay2 lander2, and only then deploys Rover 2.
- Bob then stays in the docking bay to receive sample2, and goes back to the control room before Rover 2 can transmit image2 and scan2.
- Alice starts in the docking bay, but Rover 1 has data to send, so she goes to control_room1 for its two transmissions, then comes back to the docking bay to receive sample1.

This gives four astronaut moves, two for each astronaut, as both of them need both rooms during the mission.

References

[1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, , 4th ed. Pearson, 2022. book used : link

[2] G. McGuire, B. Tugemann and G. Civario,[1201.0749] There is no 16-Clue Sudoku: Solving the Sudoku Minimum Number of Clues Problem, arXiv:1201.0749, 2012.

[3] Sudoku solving algorithms - Wikipedia

[4] Sudoku - Wikipedia

[5] P. Norvig, Solving Every Sudoku Puzzle

[6] P. Norvig, "top95.txt", 95 hard Sudoku puzzles. https://norvig.com/top95.txt

[7] Arto Inkala Sudoku - SudokuWiki.org

[8] D. McDermott, M. Ghallab, A. Howe, C. Knoblock, A. Ram, M. Veloso, D. Weld and D. Wilkins, PDDL: the Planning Domain Definition Language, version 1.2. Yale Center for Computational Vision and Control, Tech. Rep. CVC TR-98-003 / DCS TR-1165, 1998. mcdermott-et-al-tr-1998.pdf

[9] N. Lipovetzky and H. Geffner, "Best-first width search: exploration and exploitation in classical planning," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 31, no. 1, pp. 3590-3596, 2017. Best-First Width Search: Exploration and Exploitation in Classical Planning | Proceedings of the AAAI Conference on Artificial Intelligence

[10] S. Richter and M. Westphal, "The LAMA planner: guiding cost-based anytime planning with landmarks," *Journal of Artificial Intelligence Research*, vol. 39, pp. 127-177, 2010. The LAMA Planner: Guiding Cost-Based Anytime Planning with Landmarks | Journal of Artificial Intelligence Research

[11] M. Helmert, "The Fast Downward planning system," *Journal of Artificial Intelligence Research*, vol. 26, pp. 191-246, 2006. The Fast Downward Planning System | Journal of Artificial Intelligence Research

[12] J. Benton, A. Coles and A. Coles, "Temporal planning with preferences and time-dependent continuous costs," in *Proc. Int. Conf. Automated Planning and Scheduling (ICAPS)*, vol. 22, no. 1, pp. 2-10, 2012. Temporal Planning with Preferences and Time-Dependent Continuous Costs | Proceedings of the International Conference on Automated Planning and Scheduling